DD: SUB-WF-DOWNGRADE-01-FLOW-WIK — KE WIK Downgrade BPMN

Developer implementation guide: `DD_SUB-WF-DOWNGRADE-01-FLOW-WIK-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the operator BPMN flow `DD_SUB-WF-DOWNGRADE-01-FLOW-WIK-v1.0.md`.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0 Satellite of **DD_SUB-WF-DOWNGRADE-01-Subscription_Downgrade** (parent). Defines the KE WIK reference BPMN for subscription downgrade. BPMN process key: `sub-downgrade-wik`. Composes 11 workers from the framework's WORKERS catalog, 1 message catch, 1 Drools package. Includes Camunda intermediate timer for non-IMMEDIATE timings (END_OF_CURRENT_CYCLE default for KE).

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/04_subscription_workflows/DD_SUB-WF-DOWNGRADE-01-FLOW-WIK-v1.0.md`
Version	`v1.0`
DD type	`operator BPMN flow`
BPMN/process keys	`sub-downgrade-wik`
Drools packages	`rules.subscription.downgrade.wik`

4. Runtime Contracts To Implement

4.1 APIs

- `DELETE /subscriptions/sub_01HZ_OTIENO_FIBER/pending-downgrade/subop_01HZ_OTIENO_DOWNGRADE_EOC`
- `POST /api/subscriptions/sub_01HZ_OTIENO_FIBER/downgrade`

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `bil01-emit-intent`
- `compute-equipment-drop-delta`
- `drools-decide`
- `ful-call-modify`
- `fulfillment-completed`
- `fulfillment-failed`
- `kafka-emit`
- `manual-recovery-decision`
- `notification-send`
- `osr-equipment-unbind`
- `resolve-cycle-anchor-policy`
- `resolve-effective-timing`
- `sub-downgrade-wik`
- `sub-lm-commit-state`
- `sub-lm-compute-cycle-window`
- `sub-lm-put-master-fields`
- `sub-lm-put-pending-status`
- `sub-lm-revert-to-prior`
- `subscription-operation-finalize`
- `subscription-workflow-rules`
- `validate-homepass-compatibility`
- `validate-preconditions`
- `validate-target-package`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- `SubscriptionDowngradeCancelled`
- `SubscriptionDowngradeRejected`
- `SubscriptionDowngradeScheduled`
- `SubscriptionDowngraded`

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0

Satellite of **DD_SUB-WF-DOWNGRADE-01-Subscription_Downgrade** (parent). Defines the KE WIK reference BPMN for subscription downgrade. BPMN process key: `sub-downgrade-wik`. Composes 11 workers from the framework's WORKERS catalog, 1 message catch, 1 Drools package. Includes Camunda intermediate timer for non-IMMEDIATE timings (END_OF_CURRENT_CYCLE default for KE).

1. What this flow is

KE WIK's BPMN composes the framework workers to downgrade a subscription. Trigger endpoint accepts the request (from customer portal, CC, or BO admin), persists `subscription_operation` row, starts process instance of `sub-downgrade-wik`. The BPMN orchestrates: Drools validation (preconditions + target-package + equipment-drop-delta + cycle-anchor-policy + effective-timing) → if non-IMMEDIATE, emit Scheduled event + park on Camunda timer → at fire (or immediately for IMMEDIATE) re-validate → pending state flip → compute cycle window → billing intent (credit) → FUL-03 MODIFY_SERVICE → equipment-unbind step (conditional on equipment-drop delta + `unbindObsoleteEquipment` flag) → master fields update → commit state ACTIVE → emit event → notify customer → finalize.

Workers composed (11)

For non-IMMEDIATE downgrades, the BPMN inserts a **Camunda intermediate timer event** before the VALIDATING state — no extra worker needed. During the wait, `subscription_operation.current_state = SCHEDULED` and the subscription stays on the source Package.

#	Worker topic	Used for
1	`drools-decide`	Validation: preconditions, target-package, homepass-compatibility, equipment-drop delta, cycle-anchor-policy resolution, effective-timing resolution
2	`sub-lm-put-pending-status`	Flip subscription to `PENDING_DOWNGRADE`; record `current_transition_type=DOWNGRADE`, `current_transition_reason_code=<downgradeTrigger>`
3	`sub-lm-compute-cycle-window`	Compute cycle window for billing intent context
4	`bil01-emit-intent`	`DOWNGRADE_INTENT` — billing computes credit delta + any downgrade BillableEvents; settles via wallet top-up (prepaid) or invoice credit (postpaid)
5	`ful-call-modify`	FUL-03 with `intent=MODIFY_SERVICE`. FUL-03 owns the per-service-class fan-out: diff source vs target Package compositions, dispatch modifyService/deactivate per service class using HomePass `service_management_endpoints`
6	`osr-equipment-unbind`	Conditional — only if `unbindObsoleteEquipment=true` AND equipment-drop delta non-empty; marks equipment as returned-to-inventory; OSR-RMA-01 picks up downstream for physical recovery WO
7	`sub-lm-put-master-fields`	Write `package_ref`, `package_version_id`, `previous_package_ref`, `previous_package_version_id`, `last_status_changed_at`; conditional cycle-anchor mutation if RESET
8	`sub-lm-commit-state`	Flip subscription back to `ACTIVE` with new Package
9	`kafka-emit`	Emit `SubscriptionDowngradeScheduled` (acceptance) and `SubscriptionDowngraded` (commit) via outbox
10	`notification-send`	NOT-01 customer notification (reliability-isolated) at acceptance AND at commit
11	`subscription-operation-finalize`	Set `subscription_operation.final_state=COMPLETED` (or `CANCELLED` on cancellation)

Message catches

Type	Name	Used when
Intermediate timer	(timeDate=`${scheduledCommitAt}`)	Non-IMMEDIATE wait state
Boundary message	`external_cancel_requested` (correlated on `operationId`)	Cancellation during wait — fires on explicit DELETE or framework's superseding-op signal
Message catch	`fulfillment-completed` / `fulfillment-failed` (correlated on `fulfillmentCorrelationId`)	After `ful-call-modify`; BPMN waits for FUL-03

Drools package

`rules.subscription.downgrade.wik` — declared decisions:

Decision	Output	Used at
`validate-preconditions`	`eligible: bool, eligibilityReasonCode?, sourcePackageSnapshot: object`	Beginning — subscription ACTIVE, no in-flight, no pending-downgrade already, role-trigger match
`validate-target-package`	`targetValid: bool, targetReasonCode?, targetPackageSnapshot: object`	Target ACTIVE + currency match + price ≤ source + ≠ source

Decision	Output	Used at
`validate-homepass-compatibility`	`homePassCompatible: bool, homePassReasonCode?, missingServiceManagementKeys: array, unprovisionedPorts: array`	HomePass `service_management_endpoints` covers target's required service families; resolved ports non-null; HomePass status_code is_active
`compute-equipment-drop-delta`	`obsoleteServiceClasses: array, obsoleteEquipmentList: array`	Diff source vs target compositions; for each obsolete service class, identify equipment that fulfills ONLY that class (multi-service equipment retained)
`resolve-cycle-anchor-policy`	`resolvedCycleAnchorPolicy: str, policyValid: bool, policyReasonCode?`	Resolves to request value if supplied, else operator-config default; validates enum
`resolve-effective-timing`	`resolvedEffectiveTiming: str, scheduledCommitAt: timestamp?, timingValid: bool, timingReasonCode?`	Resolves to request value or operator-config default (KE: END_OF_CURRENT_CYCLE); validates `scheduledAt` bounds; computes `scheduledCommitAt`; rejects redundant END_OF_CURRENT_CYCLE + RESET combination

2. BPMN structure

Process key: `sub-downgrade-wik`. Deployed as `sub-downgrade-wik.bpmn` in the Camunda engine.

2.1 Outline

```
START (event)
  ■ Process variables from trigger endpoint:
  ■   operationId, subscriptionId, customerId, operatorCode, packageRef, homepassId,
  ■   downgradeTrigger, targetPackageRef, cycleAnchorPolicy?, effectiveTiming?, scheduledAt?,
  ■   unbindObsoleteEquipment?, notes, initiatingActorUserId, initiatingActorRole,
  ■   initiatingWorkflowKind, initiatingWorkflowRef, idempotencyKey
  ■
  ▼
[STATE: VALIDATING_REQUEST]
Service Task: drools-decide
  inputs: kjarPackage="rules.subscription.downgrade.wik",
          factsJson={subscription state, source Package snapshot, target Package snapshot,
                     target package_version snapshot, HomePass service_management_endpoints + status_code,
                     existing OSR bindings, cycleAnchorPolicy request,
                     effectiveTiming + scheduledAt request, unbindObsoleteEquipment request,
                     role, operator config},
          requestedDecisions=["validate-preconditions", "validate-target-package",
                              "validate-homepass-compatibility", "compute-equipment-drop-delta",
                              "resolve-cycle-anchor-policy", "resolve-effective-timing"]
  outputs: eligible, eligibilityReasonCode, sourcePackageSnapshot, targetValid,
           targetReasonCode, targetPackageSnapshot, homePassCompatible,
           homePassReasonCode, missingServiceManagementKeys, unprovisionedPorts,
           obsoleteServiceClasses, obsoleteEquipmentList,
           resolvedCycleAnchorPolicy, policyValid, policyReasonCode,
           resolvedEffectiveTiming, scheduledCommitAt, timingValid, timingReasonCode,
           resolvedUnbindObsoleteEquipment
  ■
  ▼ XOR: all checks pass?
  ■ NO → REJECT subflow (emit SubscriptionDowngradeRejected + finalize FAILED)
  ■ YES ↓
  ■
  ▼ XOR: resolvedEffectiveTiming == "IMMEDIATE"?
  ■ YES ↓ (go straight to commit sequence)
  ■ NO ↓
  ■   [STATE: SCHEDULED]
  ■   Service Task: kafka-emit
  ■     inputs: topic="sophix.subscription.subscription.downgradescheduled",
  ■             payloadJson=<SubscriptionDowngradeScheduled per parent §6>
  ■   ▼
  ■   Service Task: notification-send (reliability-isolated)
  ■     inputs: notificationEventKind="SUBSCRIPTION_DOWNGRADE_SCHEDULED",
  ■             eventContext={sourcePackageRef, targetPackageRef, scheduledCommitAt,
  ■                           resolvedEffectiveTiming, resolvedCycleAnchorPolicy,
  ■                           obsoleteEquipmentList}
  ■   ▼
  ■   Intermediate Catch Event: timer
  ■     timeDate: ${scheduledCommitAt}
  ■     Boundary listener: external-cancel-requested (signal from cancel REST endpoint
  ■                        OR from another state-affecting operation per R-EF-5)
  ■     → on cancel: emit SubscriptionDowngradeCancelled + finalize CANCELLED
  ■   ▼ timer fires
  ■
```

```
  ▼
[STATE: VALIDATING]
Service Task: drools-decide
  inputs: kjarPackage="rules.subscription.downgrade.wik",
          factsJson=<re-loaded current snapshots — subscription, packages, homepass, equipment>,
          requestedDecisions=["validate-preconditions", "validate-target-package",
                              "validate-homepass-compatibility", "compute-equipment-drop-delta"]
  outputs: <same shape as VALIDATING_REQUEST minus the resolved-* fields which are already set>
  ■
  ▼ XOR: revalidation passes?
  ■ NO → REJECT subflow (emit SubscriptionDowngradeRejected with rejectionStage="COMMIT_REVALIDATION")
  ■ YES ↓
  ■
[STATE: PENDING_STATE_FLIP]
Service Task: sub-lm-put-pending-status
  inputs: subscriptionId, toStatus="PENDING_DOWNGRADE",
          currentTransitionType="DOWNGRADE",
          currentTransitionReasonCode=<downgradeTrigger>
  outputs: priorStatusSnapshot, lmEventId
  ■
  ▼
Service Task: sub-lm-compute-cycle-window
  inputs: subscriptionId, referenceDate=NOW
  outputs: cycleWindowJson
  ■
  ▼
[STATE: BILLING_CALL]
Service Task: bil01-emit-intent
  inputs: subscriptionId,
          intentKind="DOWNGRADE_INTENT",
          intentContext={sourcePackageRef, sourcePackageVersionId, sourcePackagePrice,
                         targetPackageRef, targetPackageVersionId, targetPackagePrice,
                         currency, billingFrequencyChanged, downgradeTrigger,
                         cycleAnchorPolicy=<resolvedCycleAnchorPolicy>,
                         effectiveTiming=<resolvedEffectiveTiming>},
          cycleWindow
  outputs: intentDecisionId, intentDecision (lineItems with DOWNGRADE_PRORATE_CREDIT or
           DOWNGRADE_OLD_CYCLE_CREDIT + DOWNGRADE_FIRST_CYCLE per policy; totals.net ≤ 0;
           creditDistribution: {channel: "WALLET_TOPUP"|"INVOICE_CREDIT", reference})
  ■
  ▼
[STATE: FULFILLMENT_CALL]
Service Task: ful-call-modify
  inputs: subscriptionId, homepassId, customerId,
          intent="MODIFY_SERVICE",
          sourcePackageRef, sourcePackageVersionId, sourcePackageCompositionSnapshot,
          targetPackageRef, targetPackageVersionId, targetPackageCompositionSnapshot,
          homePassServiceManagementEndpoints,
          unbindObsoleteEquipment=<resolvedUnbindObsoleteEquipment>,
          fulfillmentCorrelationId=<operationId>:modify
  outputs: modifyFulfillmentId, modifyTerminalState, perServiceClassResults
  ■
  ▼ [STATE: AWAITING_FULFILLMENT_RESPONSE]
Message Catch: fulfillment-completed (correlated on fulfillmentCorrelationId)
  OR
Message Catch: fulfillment-failed
  ■ FAILED → REVERT subflow (sub-lm-revert-to-prior + emit Rejected + finalize FAILED)
  ■ COMPLETED ↓
  ■
  ▼ XOR: resolvedUnbindObsoleteEquipment AND obsoleteEquipmentList non-empty?
  ■ NO ↓
  ■ YES ↓
  ■   Service Task: osr-equipment-unbind
  ■     inputs: subscriptionId, unbindList=<obsoleteEquipmentList>,
  ■             unbindReason="DOWNGRADE_OBSOLETE"
  ■     outputs: unbindResults
  ■
  ▼
[STATE: COMMITTING_FINAL_STATE]
Service Task: sub-lm-put-master-fields
  inputs: subscriptionId,
          fieldsJson={
            package_ref: <targetPackageRef>,
            package_version_id: <targetPackageVersionId>,
            previous_package_ref: <sourcePackageRef>,
            previous_package_version_id: <sourcePackageVersionId>,
            last_status_changed_at: NOW,
            cycle_anchor_day: <if resolvedCycleAnchorPolicy=="RESET_TO_DOWNGRADE_DATE" then dayOfMonth(NOW) else unchanged
            cycle_anchor_at:  <if resolvedCycleAnchorPolicy=="RESET_TO_DOWNGRADE_DATE" then NOW else unchanged>
          }
  outputs: lmEventId
  ■
```

```
  ▼
Service Task: sub-lm-commit-state
  inputs: subscriptionId, toStatus="ACTIVE", expectedFromStatus="PENDING_DOWNGRADE"
  outputs: committedAt
  ■
  ▼
[STATE: EMITTING_EVENT]
Service Task: kafka-emit
  inputs: topic="sophix.subscription.subscription.downgraded",
          key=<subscriptionId>,
          payloadJson=<full SubscriptionDowngraded payload per parent §6>
  ■
  ▼
Service Task: notification-send (reliability-isolated, conditional on operator config)
  inputs: recipientKind="customer", recipientRef=<customerId>,
          notificationEventKind="SUBSCRIPTION_DOWNGRADED",
          eventContext={sourcePackageRef, targetPackageRef, currency,
                        cycleAnchorPolicy, resolvedEffectiveTiming,
                        creditDelta?, oldCycleCredit?, firstCycleCharge?,
                        droppedServiceClasses, unboundEquipment,
                        downgradedAt, downgradeTrigger}
  ■ failure here does NOT fail the BPMN
  ▼
[STATE: COMPLETED]
Service Task: subscription-operation-finalize
  inputs: operationId, finalState="COMPLETED"
  ■
END (event)
```

2.2 BPMN XML excerpt — timer event + conditional equipment-unbind

```
<bpmn:intermediateCatchEvent id="Catch_scheduled_commit_at" name="Wait until scheduledCommitAt">
  <bpmn:timerEventDefinition>
    <bpmn:timeDate>${scheduledCommitAt}</bpmn:timeDate>
  </bpmn:timerEventDefinition>
</bpmn:intermediateCatchEvent>

<bpmn:boundaryEvent id="Boundary_external_cancel" attachedToRef="Catch_scheduled_commit_at" cancelActivity="true">
  <bpmn:messageEventDefinition messageRef="Message_external_cancel_requested" />
</bpmn:boundaryEvent>

<bpmn:sequenceFlow sourceRef="Gateway_immediate_vs_scheduled" targetRef="Catch_scheduled_commit_at">
  <bpmn:conditionExpression>${resolvedEffectiveTiming != "IMMEDIATE"}</bpmn:conditionExpression>
</bpmn:sequenceFlow>

<bpmn:exclusiveGateway id="Gateway_equipment_unbind">
  <bpmn:outgoing>Flow_unbind_yes</bpmn:outgoing>
  <bpmn:outgoing>Flow_unbind_no</bpmn:outgoing>
</bpmn:exclusiveGateway>
<bpmn:sequenceFlow id="Flow_unbind_yes" sourceRef="Gateway_equipment_unbind" targetRef="Task_osr_unbind">
  <bpmn:conditionExpression>${resolvedUnbindObsoleteEquipment == true &amp;&amp; obsoleteEquipmentListSize &gt; 0}</bpmn:c
</bpmn:sequenceFlow>

<bpmn:serviceTask id="Task_osr_unbind" name="Unbind obsolete equipment"
                  camunda:type="external" camunda:topic="osr-equipment-unbind">
  <bpmn:extensionElements>
    <camunda:inputOutput>
      <camunda:inputParameter name="unbindReason">DOWNGRADE_OBSOLETE</camunda:inputParameter>
    </camunda:inputOutput>
  </bpmn:extensionElements>
</bpmn:serviceTask>
```

2.3 Compensation paths

If this step fails	Compensation BPMN runs
`drools-decide` validation (request OR commit revalidation)	No mutations yet; emit `SubscriptionDowngradeRejected`; finalize FAILED
Timer-fire cancellation (boundary event)	Emit `SubscriptionDowngradeCancelled`; finalize CANCELLED
`sub-lm-put-pending-status`	Atomic; failure leaves subscription ACTIVE on source Package; emit Rejected; finalize FAILED
`bil01-emit-intent`	`sub-lm-revert-to-prior` to ACTIVE; emit Rejected; finalize FAILED
`ful-call-modify` returns FAILED	`sub-lm-revert-to-prior`; emit Rejected; finalize FAILED. FUL-03 handles network-side compensation per its own rules.
`osr-equipment-unbind` failure	FUL-03 MODIFY_SERVICE already applied (obsolete services deactivated); manual recovery via `manual-recovery-decision` user task — equipment bindings out of sync with provisioned services

If this step fails	Compensation BPMN runs
`sub-lm-put-master-fields`	Manual recovery — service change applied at network level but master fields not updated
`sub-lm-commit-state`	Manual recovery
`kafka-emit`	Retried by outbox dispatcher; BPMN proceeds
`notification-send`	Logged but NOT compensated (reliability-isolated)

2.4 Configuration row (KE WIK)

In `subscription_operation_config` (framework parent):

operator_code	operation_kind	default_bpmn_process_key	operation_timeout_seconds	feature_flags
`WIK`	`DOWNGRADE`	`sub-downgrade-wik`	7776000	`{}`

(Timeout: 90 days — accommodates max_future_scheduled_days for SCHEDULED_AT case.)

In `subscription_downgrade_config` (parent §3.1):

operator_code	customer_self_service_enabled	default_cycle_anchor_policy	default_effective_timing	max_future_scheduled_days	default_unbind_obsolete_equipment	customer_notification_enabled
`WIK`	true	`PRESERVE`	`END_OF_CURRENT_CYCLE`	90	true	true

3. Drools package `rules.subscription.downgrade.wik`

KE WIK rules deployed as part of `subscription-workflow-rules` KJAR. Decisions mirror the UPGRADE-01 set with reversed price comparison (TP-3) and an equipment-DROP delta instead of equipment-ADD delta. The cycle-anchor-policy + effective-timing decisions are identical in shape to UPGRADE-01's (allowing values `RESET_TO_DOWNGRADE_DATE` vs `RESET_TO_UPGRADE_DATE`).

3.1 `validate-preconditions`

```
package rules.subscription.downgrade.wik

import com.sophix.subwf.facts.DowngradeRequestFact
import com.sophix.subwf.facts.SubscriptionStateFact
import com.sophix.subwf.facts.OperatorConfigFact

rule "Eligibility: subscription must be ACTIVE"
when
  $req: DowngradeRequestFact()
  $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId, statusCode != "ACTIVE")
then
  modify($req) { setEligible(false); setEligibilityReasonCode("INVALID_STATE_FOR_DOWNGRADE") }
end

rule "Eligibility: no in-flight operation"
when
  $req: DowngradeRequestFact(eligible == null)
  $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId, currentTransitionType != null)
then
  modify($req) { setEligible(false); setEligibilityReasonCode("OPERATION_IN_PROGRESS") }
end

rule "Eligibility: CUSTOMER_REQUESTED_DOWNGRADE requires CUSTOMER_SELF_SERVICE / CALL_CENTER / SUBSCRIPTION_OPERATOR role"
when
  $req: DowngradeRequestFact(downgradeTrigger == "CUSTOMER_REQUESTED_DOWNGRADE", eligible == null,
                             initiatingActorRole not in ("CUSTOMER_SELF_SERVICE", "CALL_CENTER", "SUBSCRIPTION_OPERATOR"))
then
  modify($req) { setEligible(false); setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER") }
end

rule "Eligibility: ADMIN_DOWNGRADE requires SUBSCRIPTION_ADMIN or SUBSCRIPTION_OPERATOR"
when
  $req: DowngradeRequestFact(downgradeTrigger == "ADMIN_DOWNGRADE", eligible == null,
                             initiatingActorRole not in ("SUBSCRIPTION_ADMIN", "SUBSCRIPTION_OPERATOR"))
then
  modify($req) { setEligible(false); setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER") }
end

rule "Eligibility: default-pass"
```

```
salience -100
when
  $req: DowngradeRequestFact(eligible == null)
then
  modify($req) { setEligible(true) }
end
```

3.2 validate-target-package

```
rule "Target: price must be ≤ source price"
when
  $req: DowngradeRequestFact(eligible == true, targetValid == null)
  $tgt: PackageFact(packageRef == $req.targetPackageRef)
  eval($tgt.getPrice().compareTo($req.getSourcePackagePrice()) > 0)
then
  modify($req) { setTargetValid(false); setTargetReasonCode("NOT_A_DOWNGRADE") }
end

# ... (status ACTIVE, currency match, ≠ source — same shape as UPGRADE-01)
```

3.3 validate-homepass-compatibility

Identical to UPGRADE-01's §3.3 — checks `service_management_endpoints` keys, ports non-null, status active.

3.4 compute-equipment-drop-delta

```
import com.sophix.subwf.facts.OsrBindingFact

rule "Equipment drop delta: identify service classes in source not in target"
when
  $req: DowngradeRequestFact(homePassCompatible == true, obsoleteServiceClasses == null)
  $src: PackageFact(packageRef == $req.sourcePackageRef)
  $tgt: PackageFact(packageRef == $req.targetPackageRef)
then
  java.util.Set obsolete = new java.util.HashSet($src.getRequiredServiceClasses());
  obsolete.removeAll($tgt.getRequiredServiceClasses());

  java.util.List obsoleteEquipment = $req.getCurrentBindings().stream()
      .filter(b -> obsolete.contains(b.getFulfillsServiceClass()))
      .filter(b -> {
        // Multi-service equipment retained: skip if it also fulfills a retained service class
        return $req.getCurrentBindings().stream()
            .filter(b2 -> b2.getEquipmentId().equals(b.getEquipmentId()))
            .noneMatch(b2 -> $tgt.getRequiredServiceClasses().contains(b2.getFulfillsServiceClass()));
      })
      .map(b -> java.util.Map.of(
          "equipmentId", b.getEquipmentId(),
          "fulfilledServiceClass", b.getFulfillsServiceClass()))
      .collect(java.util.stream.Collectors.toList());

  modify($req) {
    setObsoleteServiceClasses(new java.util.ArrayList(obsolete));
    setObsoleteEquipmentList(obsoleteEquipment)
  }
end

rule "Resolve unbindObsoleteEquipment: request override or operator default"
when
  $req: DowngradeRequestFact(resolvedUnbindObsoleteEquipment == null)
  $cfg: OperatorConfigFact()
then
  Boolean requestValue = $req.getUnbindObsoleteEquipment();
  modify($req) {
    setResolvedUnbindObsoleteEquipment(
      requestValue != null ? requestValue : $cfg.getDefaultUnbindObsoleteEquipment()
    )
  }
end
```

3.5 resolve-cycle-anchor-policy

Identical to UPGRADE-01's §3.5; enum values {`PRESERVE`, `RESET_TO_DOWNGRADE_DATE`}; default from operator config.

3.6 resolve-effective-timing

Identical to UPGRADE-01's §3.6; KE WIK operator default = `END_OF_CURRENT_CYCLE`. Rejects `END_OF_CURRENT_CYCLE` + `RESET_TO_DOWNGRADE_DATE` as redundant.

4. Worked example — Otieno's triple→double-play downgrade at end of cycle

Context: Otieno is on `pkg_INET_VOIP_IPTV_RES` (triple-play, KES 7,500/month) with an STB bound to his subscription. He calls CC on May 20 to drop IPTV and go back to `pkg_INET_VOIP_RES` (double-play, KES 4,500/month). He's fine paying through end of May; downgrade should take effect June 1. The STB will be returned (OSR-RMA-01 will dispatch a pickup WO post-commit).

May 20, 14:00:00 — Trigger endpoint:

```
POST /api/subscriptions/sub_01HZ_OTIENO_FIBER/downgrade
Idempotency-Key: downgrade-sub_01HZ_OTIENO-pkg_INET_VOIP-eoc-20260520

{ "operatorCode": "WIK", "downgradeTrigger": "CUSTOMER_REQUESTED_DOWNGRADE",
  "targetPackageRef": "pkg_01HX_INET_VOIP_RES",
  "notes": "Customer dropping IPTV — wants to keep service through May.",
  "initiatingActorUserId": "cc-agent-wik-007",
  "initiatingActorRole": "CALL_CENTER",
  "initiatingWorkflowKind": "BO_UI",
  "initiatingWorkflowRef": "cc-ticket-2026-05-20-5511" }

→ 202 Accepted
{ "operationId": "subop_01HZ_OTIENO_DOWNGRADE_EOC",
  "resolvedCycleAnchorPolicy": "PRESERVE",
  "resolvedEffectiveTiming":   "END_OF_CURRENT_CYCLE",
  "scheduledCommitAt":         "2026-05-31T23:59:59+03:00",
  "currentState":              "SCHEDULED" }
```

Request omitted `cycleAnchorPolicy`, `effectiveTiming`, `unbindObsoleteEquipment` — all resolved from KE config defaults (`PRESERVE` / `END_OF_CURRENT_CYCLE` / `true`).

May 20, 14:00:00.5 — VALIDATING_REQUEST: `drools-decide` invokes all 6 decisions. Facts: subscription ACTIVE, no in-flight, role CALL_CENTER on CUSTOMER_REQUESTED_DOWNGRADE ✓, target ACTIVE/KES/4500 ≤ 7500 ✓, HomePass `service_management_endpoints = {data, voice, iptv_multicast}` covers target's required `{data, voice}` ✓, all ports non-null, status ACT ✓. Equipment-drop delta: source has `IPTV` service class which target lacks; Otieno's current bindings: ONT (fulfills `data`), ATA (fulfills `voice`), STB (fulfills `IPTV`). STB fulfills only `IPTV` (not retained) → `obsoleteEquipmentList = [{equipmentId:"eqp_STB_01HZ_LAVI_042", fulfilledServiceClass:"IPTV"}]`. Cycle-anchor policy resolved to `PRESERVE` (config default). Effective timing resolved to `END_OF_CURRENT_CYCLE`, `scheduledCommitAt = 2026-05-31T23:59:59+03:00` (computed from cycle window). `resolvedUnbindObsoleteEquipment = true` (config default).

May 20, 14:00:01 — SCHEDULED: `kafka-emit` publishes `SubscriptionDowngradeScheduled` with `scheduledCommitAt`, `obsoleteEquipmentList`. `notification-send` → NOT-01 sends Otieno confirmation: "Your downgrade to the INET+VOIP plan is scheduled for June 1. Your STB will be collected after that date." BPMN parks on Camunda intermediate timer with `timeDate=2026-05-31T23:59:59+03:00`. Otieno's subscription stays ACTIVE on triple-play; May cycle continues to bill at KES 7,500.

May 31, 23:59:59 — Timer fires. BPMN advances to VALIDATING. Drools re-runs preconditions, target-package, homepass-compatibility, equipment-drop-delta. All pass. `sub-lm-put-pending-status` → `PENDING_DOWNGRADE`. `sub-lm-compute-cycle-window` returns new window starting June 1.

June 1, 00:00:00 — BILLING_CALL: `bil01-emit-intent` calls BIL-01 with `DOWNGRADE_INTENT`, context including `cycleAnchorPolicy=PRESERVE`, `effectiveTiming=END_OF_CURRENT_CYCLE`. BIL-01 sees commit-at-cycle-boundary case: zero credit delta (Otieno paid full month at higher tier, used full month at higher tier — no money movement). Returns `lineItems=[], totals.net=0, creditDistribution=null`.

June 1, 00:00:00 — FULFILLMENT_CALL: `ful-call-modify` calls FUL-03 with `intent=MODIFY_SERVICE`, source + target Package snapshots + HomePass `service_management_endpoints`. FUL-03 diffs compositions:

- `data` (both) → GponResProvisioner.modifyService at `{OLT-NRB-LAV-01, 0/2/03}` (no-op; no bandwidth change in this example).
- `voice` (both) → VoipSipProvisioner.modifyService at `{VOIPSWITCH-NRB-01, 5103}` (no-op).
- `iptv_multicast` (source only, DROPPED) → GponResProvisioner.deactivate at `{OLT-NRB-LAV-01, 0/2/03}` (leave multicast group, disable IGMP-snooping for Otieno's port, remove STB serial from IPTV middleware's AAA channel package).

Returns `terminalState=FULLY_FULFILLED` after 2.3 seconds.

June 1, 00:00:02.3 — AWAITING_FULFILLMENT_RESPONSE: Message catch fires. `resolvedUnbindObsoleteEquipment=true` AND `obsoleteEquipmentList` non-empty → `osr-equipment-unbind`

invoked. OSR-INSTANCE-01 marks STB `eqp_STB_01HZ_LAVI_042` as returned-to-inventory; clears the binding row.

June 1, 00:00:03 — COMMITTING_FINAL_STATE: `sub-lm-put-master-fields` writes `package_ref=pkg_01HX_INET_VOIP_RES`, `package_version_id=pkv_01HX_INET_VOIP_V3`, `previous_package_ref=pkg_01HX_INET_VOIP_IPTV_RES`, `previous_package_version_id=pkv_01HX_INET_VOIP_IPTV_V1`, `last_status_changed_at=NOW`. Cycle anchor unchanged (PRESERVE).

`sub-lm-commit-state` flips PENDING_DOWNGRADE → ACTIVE.

June 1, 00:00:03.5 — EMITTING_EVENT: `kafka-emit` publishes full `SubscriptionDowngraded` payload — including `droppedServiceClasses=["IPTV"]`, `unboundEquipment=[{equipmentId:"eqp_STB_01HZ_LAVI_042", fulfilledServiceClass:"IPTV"}]`, `creditDelta=0.00` (cycle boundary), `resolvedEffectiveTiming=END_OF_CURRENT_CYCLE`.

`notification-send` → NOT-01 confirms to Otieno: "Your plan is now the INET+VOIP package. KES 4,500/month from this month."

June 1, 00:00:04 — COMPLETED: Operation finalized.

June 1+ (downstream):

- BIL-02 consumes `SubscriptionDowngraded`; June cycle accrues at KES 4,500.
- **OSR-RMA-01 consumes** `SubscriptionDowngraded` with the `unboundEquipment` list — kicks off a customer-return WO to physically collect the STB. RMA flow lives in OSR-RMA-01's domain (pickup scheduling, customer-side return options, etc.).
- CVM analytics records the downgrade event for retention modeling.
- EM-04 reporting indexes the downgrade for monthly ARPU/churn metrics.

Otieno's subscription is now double-play. The STB binding is gone from his subscription; physical pickup is OSR-RMA-01's job.

Alternative path — IMMEDIATE downgrade with credit

If Otieno had requested `effectiveTiming=IMMEDIATE, cycleAnchorPolicy=PRESERVE` on May 20 instead:

BIL-01 sees mid-cycle commit + PRESERVE: returns `DOWNGRADE_PRORATE_CREDIT` line item for `(7500 - 4500) × 11 / 31 ≈ 1064.52` KES (credit, signed negative). BIL-01 settles per Otieno's billing mode — if prepaid, wallet top-up of KES 1,064.52; if postpaid, credit line on next invoice. FUL-03 deactivates iptv_multicast immediately. STB unbound + RMA kicked off immediately. Master fields update immediately. Total wall time ~5 seconds.

Alternative path — IMMEDIATE + RESET_TO_DOWNGRADE_DATE

BIL-01 returns two line items: `DOWNGRADE_OLD_CYCLE_CREDIT` for unused May at source price (KES 7,500 × 11 / 31 ≈ 2,661.29 credit) AND `DOWNGRADE_FIRST_CYCLE` for fresh new cycle at target price (KES 4,500 charge, cycle May 20 → June 19). Net = `-2661.29 + 4500.00 = 1838.71` (positive — Otieno pays for the new cycle but gets the old-cycle credit). Cycle anchor moves to day 20. Subsequent cycles roll from the 20th of each month.

Cancellation flow

If on May 22 Otieno had called back to cancel: `DELETE /subscriptions/sub_01HZ_OTIENO_FIBER/pending-downgrade/subop_01HZ_OTIENO_DOWNGRADE_EOC` with `cancellationReasonCode=CUSTOMER_CHANGED_MIND`. Cancel REST endpoint sends `external_cancel_requested` message correlated on operation id; Camunda boundary event on the timer fires; BPMN exits wait, emits `SubscriptionDowngradeCancelled`, finalizes as `CANCELLED`. NOT-01 confirms to Otieno. Subscription stays on source triple-play Package. No equipment-return WO is kicked off (the unbind step never ran).

5. Cross-DD coordination

Module	Direction	Contract
SUB-WF-DOWNGRADE-01 (parent)	This satellite implements parent's contract	Triggers, target-package validation, equipment-drop-delta, cycle-anchor-policy, effective-timing all defined in parent
SUB-WF-FRAMEWORK-01 (framework parent)	This satellite composes framework primitives	`subscription_operation` row written; BPMN started

Module	Direction	Contract
SUB-WF-FRAMEWORK-01-WORKERS	This satellite references workers by topic	11 workers composed
SUB-LM-01	Workers write status + master fields	Status flip + package refs + conditional cycle-anchor mutation
SIP-01	Workers read Package + Package Version catalog	Target Package validation
PLM-CFG-01	Workers read service_class definitions	Equipment-drop delta computation
RLM-CFG-01	Workers read HomePass `service_management_endpoints` + `status_code`	HomePass physical-capability check; read-only
BIL-01	`bil01-emit-intent`	`DOWNGRADE_INTENT` — credit settlement per billing mode
BIL-CFG-01	BillableEvent catalog	Downgrade-credit definitions
FUL-03	`ful-call-modify` worker	`MODIFY_SERVICE`. Owns per-service-class fan-out via HomePass service_management_endpoints
OSR-INSTANCE-01	`osr-equipment-unbind` worker	Equipment unbind
OSR-RMA-01	Consumes `SubscriptionDowngraded` event with unboundEquipment list	Kicks off equipment-return WO
NOT-01	`notification-send` worker	Customer notification per operator config
BIL-02 / CVM / EM-04	Consume Kafka events	New-price cycle accrual, churn/retention analytics, reporting

5.1 Stub debt + open coordination items

- **WUG / WTZ / YASSN FLOW satellites** — pending operator deep dives.

B — Current TZ

Status: To be filled in after codebase cartography review.

Legacy package downgrade flow exists; specific implementation shape — proration credit calculation, equipment-return propagation, network teardown sequencing, IPTV multicast group leave handling — requires source-code review. Until that cartography lands, this satellite does not assert specific legacy structural claims.